

Memory Layer als MCP Server in eurem Editor einrichten

Applied AI SS26 | HTWG Konstanz

Euer Docker Memory Container laeuft auf <http://localhost:8765/mcp> und spricht das Model Context Protocol (MCP) via Streamable HTTP. Das bedeutet: jeder MCP-kompatible Client kann sich direkt verbinden und die Memory-Tools nutzen, nicht nur smolagents.

Diese Anleitung zeigt, wie ihr den Memory Layer in verschiedenen Editoren und AI-Clients einrichtet.

Voraussetzung

Der Docker Container muss laufen. Falls nicht, siehe die Haupt-README:

```
docker start memory
```

Pruefen: `docker ps` sollte den Container `memory` auf Port 8765 anzeigen.

Cursor

Cursor unterstuetzt remote MCP Server direkt per URL.

Variante A: Projekt-Konfiguration (empfohlen, kann ins Repo committed werden)

Erstellt `.cursor/mcp.json` in eurem Projektordner:

```
{
  "mcpServers": {
    "memory": {
      "url": "http://localhost:8765/mcp"
    }
  }
}
```

Variante B: Globale Konfiguration (gilt fuer alle Projekte)

Oeffnet `~/.cursor/mcp.json` (Windows: `%USERPROFILE%\.cursor\mcp.json`) und fuegt den Server hinzu:

```
{
  "mcpServers": {
```

```
"memory": {  
  "url": "http://localhost:8765/mcp"  
}  
}
```

Nach dem Speichern: Cursor komplett beenden und neu starten. MCP Server werden nur beim Start geladen.

Pruefen: Unter **Settings > Features > MCP Servers** sollte **memory** als verbunden erscheinen.

VS Code mit GitHub Copilot

VS Code nutzt dasselbe **mcp.json**-Format, aber unter **.vscode/**.

Erstellt **.vscode/mcp.json** in eurem Projektordner:

```
{  
  "servers": {  
    "memory": {  
      "type": "http",  
      "url": "http://localhost:8765/mcp"  
    }  
  }  
}
```

Alternativ global via Command Palette: **Ctrl+Shift+P > MCP: Open User Configuration**.

Nach dem Speichern: VS Code neu starten. Die Memory-Tools tauchen dann im Copilot Chat auf, wenn ihr den Agent-Modus nutzt.

Claude Code (CLI)

```
claude mcp add --transport http memory http://localhost:8765/mcp
```

Das registriert den Server dauerhaft. Nach einem Neustart von Claude Code stehen die Memory-Tools automatisch zur Verfuegung.

Pruefen:

```
claude mcp list
```

Claude Desktop

Claude Desktop unterstützt remote MCP Server **nicht** direkt in der Konfigurationsdatei. Es gibt zwei Wege:

Variante A: Settings UI (Pro/Max/Team Plan noetig)

1. Claude Desktop oeffnen
2. Settings > Integrations (oder Connectors)
3. URL eingeben: `http://localhost:8765/mcp`
4. Fertig

Variante B: mcp-remote Bridge (funktioniert mit jedem Plan)

Oeffnet `claude_desktop_config.json`:

- Windows: `%APPDATA%\Claude\claude_desktop_config.json`
- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`

```
{
  "mcpServers": {
    "memory": {
      "command": "npx",
      "args": ["mcp-remote", "http://localhost:8765/mcp"]
    }
  }
}
```

Claude Desktop komplett beenden und neu starten (Fenster schliessen reicht nicht).

smolagents (Python)

Fuer smolagents braucht ihr keine Editor-Konfiguration. Die Verbindung passiert direkt im Code:

```
from smolagents import CodeAgent, MCPClient, InferenceClientModel

server = {"url": "http://localhost:8765/mcp", "transport": "streamable-http"}

with MCPClient(server) as tools:
    agent = CodeAgent(tools=tools, model=InferenceClientModel())
    agent.run("Speichere: Ich lerne gerade RAG")
```

Installieren: `pip install "smolagents[mcp]"`

Eigener Python-Agent (ohne smolagents)

Mit dem offiziellen `mcp` Python-SDK koennt ihr die Memory-Tools direkt aufrufen:

```
import asyncio
from mcp.client.streamable_http import streamablehttp_client
from mcp import ClientSession

async def main():
    async with streamablehttp_client("http://localhost:8765/mcp") as (read, write,
_):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Tools auflisten
            tools = await session.list_tools()
            for tool in tools.tools:
                print(f" {tool.name}")

            # Fakt speichern
            await session.call_tool("memory_store", {
                "content": "RAG kombiniert Retrieval und Generation",
                "metadata": {"tags": "aai,rag,konzept"}
            })

            # Semantisch suchen
            result = await session.call_tool("memory_search", {
                "query": "Wie funktioniert RAG?"
            })
            print(result.content[0].text)

asyncio.run(main())
```

Installieren: `pip install mcp httpx`

Welche Tools stehen zur Verfuegung?

Unabhaengig vom Client habt ihr immer dieselben Tools:

Tool	Beschreibung
<code>memory_store</code>	Fakt/Erkenntnis speichern (mit optionalen Tags)
<code>memory_search</code>	Semantische Suche ueber gespeicherte Memories
<code>memory_list</code>	Alle Memories auflisten
<code>memory_delete</code>	Memory loeschen (per Hash)
<code>memory_update</code>	Metadata einer Memory aktualisieren
<code>memory_health</code>	Healthcheck des Servers

Tool	Beschreibung
<code>memory_stats</code>	Server-Statistiken
<code>memory_graph</code>	Assoziationen zwischen Memories erkunden

Fehlerbehebung

Problem	Loesung
Server nicht erreichbar	<code>docker ps</code> pruefen, ob Container laeuft
Tools tauchen nicht auf	Editor komplett neu starten (nicht nur Fenster schliessen)
"Connection refused"	Port 8765 pruefen: <code>curl -X POST http://localhost:8765/mcp</code>
Claude Desktop zeigt nichts	mcp-remote installiert? <code>npx mcp-remote --version</code>